# Design Controller for an Artificial Neural Network (ANN) Accelerator and its Peripherals

User Manual

A SystemVerilog Controller for a Y-Flash Memristor ANN core

---


**Team:** Mohammad Abu Ammar & Lana Bakrieh

**Supervisor:** Nir Ben-Haim

**Institution:** Technion — VLSI Laboratory


---

June 30, 2026

# Contents

# 1 Project Goal & Overview

The **ANN Accelerator Controller** is a digital control unit, written in SystemVerilog, that drives an Analog Neural Network (ANN) implemented on a **Y-Flash memristor crossbar array**. It acts as the bridge between a host processor and the analog in-memory-computing core.

The controller accepts five commands from a host through a parallel interface and translates them into precisely timed *pulse trains* that the analog array requires:

- **Programming** a quantized weight into a selected memristor cell.
- **Verifying** the programmed value with a closed feedback loop and automatically re-programming or erasing on mismatch.
- **Erasing** a cell.
- **Reading** a stored weight back from the array.
- **Inference**: collecting an $8 \times 8$ input image into an input buffer and streaming it bit-serially into the array for matrix–vector multiplication.

The central design challenge — and the project's main contribution — is the **closed-loop program/verify algorithm**. Because memristor programming is analog and imprecise, the controller reads back each weight after programming. If the cell is *under-programmed*, it re-applies program pulses (up to `MAX_PROG_RETRIES`); if it is *over-programmed*, it erases the cell and retries. A lookup table (LUT) sets the number of program-pulse repetitions per target weight value, allowing weight-dependent pulse shaping.

# 2 Architecture Components

The system consists of three RTL modules and three supporting packages. The top-level module is `ann_controller`.

| Role | Module name | Source file |
|---|---|---|
| Parallel Interface | `parallel_interface` | `rtl/Parallel_Interface/parallel_interface.sv` |
| ANN Controller (top) | `ann_controller` | `rtl/Controller/controller.sv` |
| Input Buffer | `input_buffer` | `rtl/Input_Buffer/input_buffer.sv` |

## 2.1 Parallel Interface (`parallel_interface`)

Decodes the 32-bit host word `host_data` into an 8-bit data byte and a 16-bit address, passes through the 3-bit command `host_cmd`, and asserts `valid` only when the command is active and the one-hot address tail is well formed. The host word layout is `host_data[31:24]` = data byte and `host_data[23:0]` = one-hot {PE, SA, column, row}.

## 2.2 ANN Controller (`ann_controller`)

The heart of the system. It implements a 9-state main FSM (`S_IDLE, S_RESET, S_PROGRAM, S_VERIFY, S_ERASE, S_READ, S_COLLECT_DATA, S_COMPUTE, S_RESULT`) plus three sub-FSMs for the `PROGRAM`, `VERIFY`, and `ERASE` sequences. It generates the 3-bit `pulses` mode bus and the packed 32-bit `ann_address` (data byte + one-hot address) toward the analog core, and orchestrates the input buffer during inference.

### 2.3 Input Buffer (`input_buffer`)

A 64 × 8-bit memory. During inference it is loaded with an 8 × 8 image; during compute it outputs eight channels `D0-D7`, one bit per channel per cycle (bit-serial, LSB-first). During weight verification it returns the full byte at the addressed location.

## 3 Parameter Configuration

All defaults are defined in `controller_pkg` and the supporting packages. The top module is parameterizable:

```
module ann_controller #(
    parameter int ADDR_WIDTH            = controller_pkg::DEFAULT_ADDR_WIDTH,
        // 8
    parameter int WEIGHT_WIDTH          = controller_pkg::DEFAULT_WEIGHT_WIDTH,
        // 16
    parameter bit USE_WEIGHT_PULSE_LUT  = 1'b1,
    parameter string WEIGHT_PULSE_LUT_FILE =
        "target/Controller/programming_inputs/weight_pulse_lut.mem"
) ( ... );
```

| Parameter | Default | Description |
|---|---|---|
| `ADDR_WIDTH` | 8 | Host address width (default). |
| `WEIGHT_WIDTH` | 16 | Weight data width (default). |
| `USE_WEIGHT_PULSE_LUT` | 1 | Enable LUT-driven program-pulse repeat count. |
| `WEIGHT_PULSE_LUT_FILE` | *(path)* | `$readmemh` LUT file: weight → repeat count. |
| **Address map (`controller_pkg`, total `WEIGHT_ADDR_WIDTH`=10)** | | |
| `BLOCK_ID_WIDTH` | 2 | PE / block select (`address[9:8]`). |
| `SUB_BLOCK_ID_WIDTH` | 2 | Sub-array select (`address[7:6]`). |
| `COL_ID_WIDTH` | 3 | Column within sub-block (`address[5:3]`). |
| `ROW_ID_WIDTH` | 3 | Row within sub-block (`address[2:0]`). |
| **Pulse generation (`controller_pkg`)** | | |
| `TREAD` | 2 | READ burst width (clock cycles). |
| `TPROG` | 2 | PROGRAM burst width. |
| `TERASE` | 2 | ERASE burst width. |
| `TINF` | 8 | INFERENCE burst width (min 8 for bit-serial). |
| `PULSE_GAP` | 1 | Idle (HiZ) cycles between bursts. |
| `PULSE_NUM_READ` | 1 | Number of READ bursts per operation. |
| `PULSE_NUM_PROG` | 1 | Number of PROGRAM bursts per operation. |
| `PULSE_NUM_ERASE` | 1 | Number of ERASE bursts per operation. |
| `PULSE_NUM_INF` | 1 | Number of INFERENCE bursts per operation. |
| `MAX_PROG_RETRIES` | 3 | Re-PROG attempts before falling back to ERASE. |
| **Buffer / array sizing** | | |
| `BUFFER_SIZE` | 64 | Input-buffer depth (8 × 8 image). |
| `BUFFER_DATA_WIDTH` | 8 | Buffer word width. |
| `TOTAL_WEIGHT_LOCATIONS` | 1024 | 4 × 4 × 8 × 8 array cells. |
| `DEFAULT_NUM_WEIGHTS` | 640 | Default 10 × 64 weight matrix. |

# 4 Top-Level I/O Interface

The following table lists every port of the top-level module `ann_controller` with its exact direction, bit-width, and function inferred from the RTL.

Table 2: `ann_controller` port list

| Signal | Dir | Width | Description |
|---|---|---|---|
| **Global** | | | |
| `clk` | In | 1 | System clock. |
| `rst_n` | In | 1 | Active-low asynchronous reset. |
| **Controller ↔ Parallel Interface** | | | |
| `valid` | In | 1 | Asserted when a command/data beat is ready. |
| `data` | In | 8 | Data byte from the parallel interface (weight/pixel payload). |
| `address` | In | 16 | Decoded address {reserved[15:10], block[9:8], sub_block[7:6], col[5:3], row[2:0]}. |
| `cmd` | In | 3 | Command (`CMD_HIZ/READ/PROG/ERASE/INF`). |
| **Controller ↔ ANN Core** | | | |
| `ann_reset` | Out | 1 | Reset pulse toward the ANN core (asserted in `S_RESET`). |
| `op_done` | In | 1 | Core handshake: advances PROG/ERASE sub-FSMs. |
| `ann_address` | Out | 32 | Packed bus: [31:24] data byte, [23:0] one-hot {PE,SA,col,row}. |
| `pulses` | Out | 3 | Pulse-mode bus (HIZ/READ/PROG/ERASE/INF) gated by burst timing. |
| `weight_read_data` | In | 4 | Quantized weight read back from the memristor during VERIFY. |
| **Controller ↔ Input Buffer** | | | |
| `buf_reg_add` | Out | 6 | Buffer address. |
| `buf_reg_ctrl` | Out | 3 | Buffer control (`CTRL_IDLE/DATA_LOAD/COMPUTE/RESULT_OUT/WEIGHT_READ`). |
| `buf_read_write` | Out | 1 | 1 = write to buffer, 0 = read. |
| `buf_bit_sel` | Out | 3 | Bit index 0–7 for bit-serial D0–D7 output (LSB-first). |
| `buf_data_out` | Out | 8 | Data written into the buffer (from host / parallel interface). |
| `buf_ready` | In | 1 | Buffer ready/valid flag. |
| `buf_data` | In | 8 | Full byte read from the buffer at the current address. |
| **Status** | | | |
| `busy` | Out | 1 | High whenever the controller is not in `S_IDLE`. |

### 4.1 Command Encoding

| Command | cmd[2:0] | Operation |
|---|---|---|
| CMD_HIZ | 000 | Idle / high-impedance, no pulses. |
| CMD_READ | 001 | Read a weight from the addressed cell. |
| CMD_PROG | 010 | Program a weight (LUT-shaped pulse train) then auto-verify. |
| CMD_ERASE | 011 | Erase the addressed cell. |
| CMD_INF | 100 | Collect $8 \times 8$ data and run bit-serial inference. |

## 5 Execution & Simulation Guide

The project is driven by `scripts/run_sim.py`, which wraps ModelSim's `vlog` (compile) and `vsim` (simulate). Run all commands from the project root.

### 5.1 Prerequisites

- ModelSim / QuestaSim ASE (Intel FPGA 17.0) with `vlog`/`vsim` on `PATH`.
- Python 3.8+ (plus `numpy`, `scikit-learn` for stimulus generation).

### 5.2 List available modules and testbenches

```
python scripts/run_sim.py list
```

### 5.3 Compile

```
# RTL only
python scripts/run_sim.py compile -m Controller -t rtl
# Full verification set (RTL + testbenches)
python scripts/run_sim.py compile -m Controller -t verif
```

### 5.4 Run a single testbench (console / batch)

```
python scripts/run_sim.py sim -m Controller -tb controller_prog_verify_lut_tb
```

### 5.5 Run with GUI + waveforms

Passing `-do-file` opens the ModelSim GUI and loads the wave layout. Use the `*_waves_tb` wrapper testbench with its matching `.do`:

```
python scripts/run_sim.py sim -m Controller \
  -tb controller_prog_verify_lut_tb_waves_tb \
  --do-file tb/Controller/do/waves/controller_prog_verify_lut_tb_waves.do
```

### 5.6 Run all testbenches for a module

```
python scripts/run_sim.py run_all -m Controller
python scripts/run_sim.py run_all -m Input_Buffer -m Parallel_Interface
```

### 5.7 Clean generated artifacts

```
python scripts/run_sim.py clean -a % work/ + target/ outputs
python scripts/run_sim.py clean -w % work/ only
python scripts/run_sim.py clean -t -m Controller
```

### 5.8 Detailed runbooks

Per-testbench compile/run commands (batch and GUI wave) are maintained under `docs/verification/`:

- `TESTBENCH_CATALOG.md` — what each testbench checks
- `REGULAR_TESTBENCH_RUNBOOK.md` — batch (`vsim -c`) runs
- `WAVE_TESTBENCH_RUNBOOK.md` — GUI + `-do-file` wave runs

### 5.9 Available testbenches

| Module | Testbenches |
|---|---|
| Controller | `controller_addr_pulse_tb`, `controller_prog_verify_lut_tb`, `controller_prog_verify_lut_10w_tb`, `parallel_interface_controller_integration_tb`, `controller_host_erase_tb`, `controller_inf_buffer_flow_tb`, `controller_host_read_reorder_tb` |
| Input_Buffer | `input_buffer_bit_serial_tb`, `input_buffer_reset_behavior_tb`, `input_buffer_full_overwrite_tb` |
| Parallel_Interface | `parallel_interface_extract_tb` |

**Output location.** Reports and logs are written under `target/<module>/` (e.g. `target/Controller/prog/prog_verify_report.txt`).